



HeeraMon

Notification & Background-Agent Specification

Background Agent Specification Document

Date of Preparation: September 30, 2026

Document ID	HM-DOC-12
Title	Notification & Background-Agent Specification
Classification	Background Agent Specification Document
Status	Draft
Version	1.0
Owner	Product / Engineering
Audience	All project contributors
Related Documents	Technical Architecture (HM-DOC-07), Matching Spec (HM-DOC-06)

Contents

1 Purpose	2
2 Background Agent Responsibilities	2
3 Search Scheduling	2
4 Background Task Lifecycle	2
5 Match Detection	3
6 Notification Principles	3
7 Notification Channels	4
8 Notification Preferences	4
9 Notification Deduplication	4
10 Failures and Retries	4
11 User Control	5
12 Agent Autonomy Boundary	5
13 Observability	6
14 Boundaries	6
15 Success Criteria	6
16 Document Control	7

1 Purpose

This document defines how HeeraMon performs work while the user is away and how meaningful events are communicated to the user.

The goal is to make job seeking continuous without becoming noisy.

2 Background Agent Responsibilities

The background system may perform tasks such as:

- checking configured job sources;
- ingesting new jobs;
- normalizing and deduplicating jobs;
- evaluating relevant jobs against user preferences;
- identifying meaningful matches;
- updating job status;
- generating notifications;
- retrying failed operations.

Background processing must not require the user's browser to remain open.

3 Search Scheduling

Each user should have a configurable search frequency.

Possible options:

- manual;
- once daily;
- several times per day;
- other predefined intervals supported by infrastructure.

The system should avoid unnecessarily frequent searches when no meaningful change is expected.

Search schedules should be stored independently from the frontend so that background execution continues even when the user is offline.

4 Background Task Lifecycle

A background task should generally follow:

Scheduled → Running → Completed / Failed → Retry if appropriate

Tasks should be designed to be idempotent where possible.

Repeated execution must not create:

- duplicate jobs;
- duplicate matches;
- duplicate notifications;
- duplicate applications.

5 Match Detection

A background search should not notify the user about every job it encounters.

HeeraMon should notify when an opportunity is sufficiently meaningful according to the user's preferences and previous interaction.

Examples:

- strong match;
- close match with an important trade-off;
- newly discovered job that satisfies a previously unmet hard requirement;
- important change to an already relevant job.

The exact notification threshold may evolve as matching improves.

6 Notification Principles

Notifications should be:

- useful;
- concise;
- understandable;
- actionable;
- non-intrusive.

A notification should communicate why the user should care.

Example:

"I found a 45k AI/ML role that matches your main requirements. The commute is about 50 minutes instead of your preferred 30."

Rather than:

"New job found."

7 Notification Channels

The system may support:

- in-app notifications;
- email notifications.

Additional channels such as browser push or mobile push can be introduced later.

The architecture should allow notification channels to be added without changing the matching system.

8 Notification Preferences

Users should eventually be able to control:

- whether notifications are enabled;
- preferred notification channels;
- search frequency;
- notification density;
- potentially the types of events that trigger notifications.

The system should avoid overwhelming users with repetitive notifications.

9 Notification Deduplication

The same opportunity should not repeatedly notify the user unless something meaningful has changed.

The system should track notification history and relevant job/match state.

Examples of potentially meaningful changes:

- salary changed;
- job status changed;
- new information became available;
- job became a significantly better match;
- application status changed.

10 Failures and Retries

Background operations should handle temporary failures gracefully.

Examples:

- job source unavailable;

- LLM provider unavailable;
- notification provider unavailable;
- database/network timeout.

The system should:

- retry appropriate transient failures;
- avoid infinite retries;
- record failure state;
- continue processing independent tasks;
- notify the user only when a failure materially affects them.

11 User Control

Users should be able to:

- pause continuous searching;
- resume searching;
- change search frequency;
- review recent search activity where useful;
- control notification preferences.

Pausing notifications should not necessarily stop background searching unless the user explicitly chooses to pause searching itself.

12 Agent Autonomy Boundary

Background HeeraMon may independently:

- search;
- ingest;
- analyze;
- compare;
- learn from confirmed interactions;
- notify.

It must not independently:

- submit applications;
- change important user requirements;
- disclose private information;
- perform other consequential external actions without the required authorization.

13 Observability

Background tasks should record sufficient operational information to determine:

- what task ran;
- when it ran;
- whether it succeeded;
- what failed;
- whether it was retried;
- whether a notification was generated.

Logs should avoid unnecessary storage of private user content.

14 Boundaries

The initial release does not require:

- real-time job monitoring;
- complex event-streaming infrastructure;
- mobile push notifications;
- sophisticated notification personalization;
- autonomous application submission;
- unlimited search frequency.

A reliable scheduled background worker is sufficient.

15 Success Criteria

The background-agent system is successful when HeeraMon can:

1. Continue searching while the user is offline.
2. Run searches according to user-controlled frequency.
3. Process newly discovered jobs.
4. Identify meaningful opportunities.
5. Notify the user without unnecessary noise.
6. Retry temporary failures safely.
7. Avoid duplicate jobs and notifications.
8. Preserve the human approval boundary for consequential actions.

Core principle: HeeraMon should work in the background, but should only interrupt the user's life when it has something genuinely worth bringing to their attention.

16 Document Control

Version	Date	Author / Role	Changes
1.0	September 30, 2026	Product / Engineering	Initial release

Approval: This document defines the notification and background-agent behavior for HeeraMon. All subsequent design and implementation work must remain consistent with the principles stated herein.

HeeraMon flies to help people find job that fits not only their career, but the life they want to live.



Saadman Sakib
Product Engineer